REST ASSURED • CLASS 9

Builder Pattern, Nested POJOs & Date/Time Serialization

Beginner-friendly complete notes with code walkthroughs, realistic scenarios, exercises, solutions and best practices

Based on your uploaded "Rest Assured class 9.txt"

Learning target By the end, you should be able to model a nested product JSON payload using POJOs, construct it with Lombok's @Builder, serialize it correctly with Jackson, send it through Rest Assured, and validate the response.

How to use these notes

Read the examples in order and type them into your own Maven project. The guide follows the source material's product/category payload, builder examples and timestamp fields. Where the original snippet contains compile-time or design problems, the correction is called out explicitly rather than silently treating the source as runnable.

Learning roadmap

1. Understand the JSON contract and how nested JSON maps to Java classes.
2. Create Products and Category POJOs with Lombok annotations.
3. Compare setter-based construction with the Builder pattern.
4. Understand serialization and deserialization with Jackson.
5. Handle timestamps carefully: LocalDateTime versus OffsetDateTime.
6. Send the request with Rest Assured and assert meaningful response fields.
7. Complete the hands-on tasks and troubleshooting checklist.

Contents

Section	What it covers
1	Read the sample product JSON
2	POJO classes and Lombok annotations
3	Builder pattern explained
4	Nested builder: Products + Category
5	Serialization and deserialization
6	Date/time modelling and Jackson configuration
7	Corrected end-to-end Rest Assured example
8	Problems in the supplied source and why they matter
9	Real-world scenarios and negative tests
10	Best practices, troubleshooting and exercises

1. Understand the sample product JSON

The source payload represents a product. Some fields are simple values, while category is a nested object and images is an array of strings. Product and category also contain created/updated timestamps.

```
{
  "title": "Sleek Modern Laptop with Ambient Lighting",
  "slug": "sleek-modern-laptop-with-ambient-lighting",
  "price": 43,
  "description": "Experience next-level computing",
  "category": {
    "id": 2,
    "name": "Electronics",
    "slug": "electronics",
    "image": "https://i.imgur.com/ZANVnHE.jpeg",
    "creationAt": "2026-09-29T09:08:39.000Z",
```

```
    "updatedAt": "2026-09-29T09:08:39.000Z"
  },
  "images": [
    "https://i.imgur.com/OKn1KFI.jpeg",
    "https://i.imgur.com/G4f21Ai.jpeg",
    "https://i.imgur.com/Z9oKRVJ.jpeg"
  ],
  "creationAt": "2026-09-29T09:08:39.000Z",
  "updatedAt": "2026-09-29T09:08:39.000Z"
}
```

Map JSON fields to Java fields

JSON part	Example value	Java representation	Reason
title	text	String	A single text value.
price	43	Integer or BigDecimal	Choose based on the API's contract; BigDecimal is safer for monetary arithmetic.
category	JSON object	Category	A nested object gets its own POJO.
images	array of strings	List<String>	A collection of zero or more image URLs.
creationAt	timestamp text	OffsetDateTime / Instant recommended	The Z suffix indicates UTC offset/time-zone semantics.
slug	text	String	A URL-friendly product identifier.

Key idea Your Java object structure should mirror the JSON structure. If JSON has category.id, your Products object should hold a Category object; you do not need a separate flat field called categoryId unless the API contract uses one.

What the API contract means

The API contract is the agreed shape and meaning of the request/response: field names, required fields, data types, allowed values, status codes and timestamp format. In real projects, use the Swagger/OpenAPI specification or an approved example payload to design POJOs; do not guess field names.

2. Create POJO classes

POJO means Plain Old Java Object. It is a normal Java class used to represent data. For API testing, request POJOs give the test a structured way to build JSON without manually concatenating strings.

Lombok prerequisite

The @Data and @Builder annotations are generated during compilation. Add Lombok to the Maven project and enable annotation processing in the IDE/build if your editor reports that generated methods do not exist. Use a version compatible with your Java and build-tool setup.

```
<dependency>
  <groupId>org.projectlombok</groupId>
```

```
    <artifactId>lombok</artifactId>
    <version>${lombok.version}</version>
    <scope>provided</scope>
</dependency>
```

Products.java

```
import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.List;
import lombok.Builder;
import lombok.Data;

@Data
@Builder(toBuilder = true)
public class Products {
    private String title;
    private String slug;
    private BigDecimal price;
    private String description;
    private Category category;
    private List<String> images;
    private OffsetDateTime creationAt;
    private OffsetDateTime updatedAt;
}
```

Category.java

```
import java.time.OffsetDateTime;
import lombok.Builder;
import lombok.Data;

@Data
@Builder(toBuilder = true)
public class Category {
    private Integer id;
    private String name;
    private String slug;
    private String image;
    private OffsetDateTime creationAt;
    private OffsetDateTime updatedAt;
}
```

Line-by-line concepts

Code	Meaning
@Data	Lombok generates getters, setters, toString, equals and hashCode methods for the class.
@Builder	Lombok generates builder methods such as Products.builder().title(...).price(...).build().
@Builder(toBuilder = true)	Also generates an instance toBuilder() method. Useful for copying a fixture and changing a few fields.
private Category category	Products contains one Category object as a nested property.

Code	Meaning
List<String> images	The property contains multiple text values; Jackson writes it as a JSON array.
OffsetDateTime	Represents date/time with an offset, such as UTC Z, rather than discarding the offset.

Note about types The source code uses Integer for price and LocalDateTime for timestamps. Those types can be valid when they match the actual API contract. This guide uses BigDecimal for prices and OffsetDateTime for timestamps because the example JSON contains a decimal-friendly money field and a Z timezone marker. Keep the types aligned with your real API contract.

3. The Builder pattern

The Builder pattern constructs an object by naming each value in a chain of method calls. It is especially helpful when a request has many fields, optional fields, nested objects or lists.

Traditional setter approach

```
Products product = new Products();
product.setTitle("iPhone 17");
product.setSlug("iphone-17");
product.setPrice(new BigDecimal("1000.00"));
product.setDescription("iPhone 17 promotion");
product.setImages(List.of(
    "https://shop.example.test/images/iphone-front.png",
    "https://shop.example.test/images/iphone-back.png"
));
```

This style is straightforward, but it needs many lines. It also creates an object before it has been populated, so a partially configured object can accidentally be used.

Builder approach

```
Products product = Products.builder()
    .title("iPhone 17")
    .slug("iphone-17")
    .price(new BigDecimal("1000.00"))
    .description("iPhone 17 promotion")
    .images(List.of(
        "https://shop.example.test/images/iphone-front.png",
        "https://shop.example.test/images/iphone-back.png"
    ))
    .build();
```

Understand the chain

Expression	What happens
Products.builder()	Creates a builder object for Products.
.title("iPhone 17")	Sets the title value on the builder.

Expression	What happens
.price(new BigDecimal("1000.00"))	Sets price; values can be read easily because the field name is explicit.
.images(List.of(...))	Supplies the image list.
.build()	Creates the final Products instance.

Important The builder does not send a request and does not create JSON by itself. It only constructs a Java object. Jackson/Rest Assured later converts that object to a JSON request body.

When to use a Builder

- Many fields or several optional request fields.
- Nested objects that also need to be built, such as Product → Category.
- Tests where each test needs a slightly different payload.
- Request objects where named assignments improve readability.

For a very small DTO with one or two fields, setters or a constructor may be simpler. Use the Builder because it improves the test code, not merely because it exists.

4. Build nested objects correctly

The source JSON contains category inside product. Build Category first, then pass the finished Category object into Products.builder().

```
OffsetDateTime createdAt = OffsetDateTime.parse("2026-09-29T09:08:39Z");
OffsetDateTime updatedAt = OffsetDateTime.parse("2026-09-29T09:08:39Z");

Category category = Category.builder()
    .id(2)
    .name("Electronics")
    .slug("electronics")
    .image("https://i.imgur.com/ZANVnHE.jpeg")
    .creationAt(createdAt)
    .updatedAt(updatedAt)
    .build();

Products product = Products.builder()
    .title("Sleek Modern Laptop with Ambient Lighting")
    .slug("sleek-modern-laptop-with-ambient-lighting")
    .price(new BigDecimal("43.00"))
    .description("Experience next-level computing")
    .category(category)
    .images(List.of(
        "https://i.imgur.com/OKn1KFI.jpeg",
        "https://i.imgur.com/G4f21Ai.jpeg",
        "https://i.imgur.com/Z9oKRVJ.jpeg"
    ))
    .creationAt(createdAt)
    .updatedAt(updatedAt)
    .build();
```

Object hierarchy

```
Products
 ├── title: String
 ├── slug: String
 ├── price: BigDecimal
 ├── category: Category
 │    ├── id: Integer
 │    ├── name: String
 │    ├── slug: String
 │    └── image: String
 ├── images: List<String>
 ├── creationAt: OffsetDateTime
 └── updatedAt: OffsetDateTime
```

Common beginner trap: null nested object

```
Products product = new Products();
product.getCategory().setId(101); // Can throw NullPointerException
```

Why? Creating Products does not automatically create the Category inside it. The category field may still be null. Either construct a Category and call product.setCategory(category), or use nested builders as shown above.

5. Serialization and deserialization

Term	Definition	Example in this lesson
Serialization	Converts a Java object into a JSON string/byte representation.	ObjectMapper.writeValueAsString(product)
Deserialization	Converts JSON input into a Java object.	ObjectMapper.readValue(file, Products.class)
Builder	Creates/populates a Java object with named values.	Products.builder().title(...).build()
HTTP request body	The representation sent to the endpoint.	request.body(jsonPayload)

Serialization

```
ObjectMapper mapper = new ObjectMapper();
String jsonPayload = mapper.writeValueAsString(product);
System.out.println(jsonPayload);
```

writeValueAsString serializes the Java object. It does not make a network call. The returned String can then be supplied to Rest Assured.

Deserialization

```
File payloadFile = new File("src/test/resources/payload/product.json");
Products product = mapper.readValue(payloadFile, Products.class);
```

readValue reads JSON and maps the JSON properties into a Products instance. This is useful when a team stores a stable baseline request in a fixture file and wants to update only a few fields per test.

Fixture + builder for dynamic variations

```
File payloadFile = new File("src/test/resources/payload/product.json");
Products fixture = mapper.readValue(payloadFile, Products.class);

Products changedProduct = fixture.toBuilder()
   .title("Laptop generated for regression test")
   .slug("laptop-regression-001")
   .price(new BigDecimal("849.99"))
   .build();

String requestJson = mapper.writeValueAsString(changedProduct);
```

This copy-and-change approach retains fields loaded from the fixture unless they are replaced. It works only when the class has @Builder(toBuilder = true) and the JSON fixture can be deserialized into the POJO.

6. Date/time handling with Jackson

The sample payload uses strings such as 2026-09-29T09:08:39.000Z. The Z means UTC. Date/time types are not interchangeable; choose the Java type to reflect whether the incoming text carries a time-zone offset.

Java type	Example input	Use when
LocalDateTime	2026-09-29T09:08:39	The value represents a local wall-clock date/time with no offset or zone.
OffsetDateTime	2026-09-29T09:08:39Z or 2026-09-29T12:08:39+03:00	The API provides a date/time with an offset, including UTC Z.
Instant	2026-09-29T09:08:39Z	The value represents a point on the UTC timeline; good for audit timestamps.

Why the original pattern is risky A LocalDateTime has no zone/offset. A pattern cannot make it retain a UTC offset. Because the provided sample ends with Z and has fractional seconds .000, use OffsetDateTime or Instant for accurate semantics, or change the contract to a timezone-free LocalDateTime string if that is what the API actually requires.

Recommended Jackson setup for java.time

For Jackson 2.x, include the Java Time module and register it. Match the module version to the Jackson core/databind version already used in the project.

```
// Maven dependency: keep version aligned with jackson-databind
<dependency>
   <groupId>com.fasterxml.jackson.datatype</groupId>
   <artifactId>jackson-datatype-jsr310</artifactId>
   <version>${jackson.version}</version>
</dependency>
```

7. Corrected end-to-end Rest Assured example

This example creates a nested product payload and posts it to the endpoint used in the source file. Confirm that this endpoint is available and that its current contract accepts these fields before running it. Status code 201 below is an expected example for resource creation; change it if the target API contract specifies another success code.

```
import static org.hamcrest.Matchers.equalTo;
import static org.hamcrest.Matchers.notNullValue;
import static org.hamcrest.Matchers.greaterThan;

import java.math.BigDecimal;
import java.time.OffsetDateTime;
import java.util.List;
import org.testng.annotations.Test;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;

public class ProductCreateTest {
   @Test
   public void createProduct() throws Exception {
      ObjectMapper mapper = new ObjectMapper();
      mapper.registerModule(new JavaTimeModule());
      mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);

      OffsetDateTime timestamp = OffsetDateTime.parse("2026-09-29T09:08:39Z");
      Category category = Category.builder()
         .id(2).name("Electronics").slug("electronics")
         .image("https://i.imgur.com/ZANVnHE.jpeg")
         .creationAt(timestamp).updatedAt(timestamp).build();

      Products product = Products.builder()
         .title("Regression Laptop").slug("regression-laptop-001")
         .price(new BigDecimal("43.00"))
         .description("Product created by API automation")
         .category(category)
         .images(List.of("https://i.imgur.com/OKn1KFI.jpeg",
                  "https://i.imgur.com/G4f21Ai.jpeg"))
         .creationAt(timestamp).updatedAt(timestamp).build();

      String payload = mapper.writeValueAsString(product);
```

Continue inside createProduct(): send and validate the request

```
Response response = RestAssured.given()
   .contentType(ContentType.JSON)
   .accept(ContentType.JSON)
   .body(payload)
   .when()
   .post("https://api.escuelajs.co/api/v1/products");
response.then().log().ifValidationFails()
   .statusCode(201)
   .body("id", notNullValue())
   .body("title", equalTo("Regression Laptop"))
   .body("price", greaterThan(0));
   }
}
```

Walkthrough of the test

- Configure ObjectMapper to support java.time values before serializing the POJO.
- Build Category first so the nested category object is not null.
- Build Products and attach the category and image list.
- Call writeValueAsString to get the JSON representation.
- Use ContentType.JSON so the request advertises JSON content.
- Send POST to the target endpoint.
- Assert the expected creation status code and important response fields. Logging only on validation failures reduces noise and avoids routinely printing sensitive content.

Test framework note This example uses TestNG for the @Test annotation. If your project uses JUnit 5 instead, replace the import and configure the Maven/test runner consistently. Do not mix annotations casually.

8. Problems in the supplied source and why they matter

Original issue	Why it fails / risk	Correction
products.getCategory().setId(...) immediately after new Products()	category may be null; calling a method on null throws NullPointerException.	Build Category first and set it on Products, or use nested builder calls.
Two variables named Products products in one method	Duplicate local variable declaration is a compile-time error.	Keep one construction approach in a method, or use different variable names / separate methods.
getCatogory() typo	No matching getter exists if the field is named category.	Use getCategory() or builder field .category(...).
setCreationDate(...) while field is creationAt	Generated method name comes from field name; this setter may not exist.	Use setCreationAt(...) or .creationAt(...).
LocalDateTime.of(..., 0, ZoneOffset.UTC)	The argument list does not match LocalDateTime.of overloads; LocalDateTime has no zone parameter.	Use LocalDateTime.of(y,m,d,h,min,sec) without zone, or OffsetDateTime with offset-aware data.
OffZoneset.UTC typo	The symbol does not exist.	Use ZoneOffset.UTC only with an offset-aware type such as OffsetDateTime.
Trailing comma in updatedAt(LocalDateTime.of(...,))	Invalid Java syntax.	Remove the trailing comma and use a valid method signature.
@JsonFormat pattern excludes fractional seconds or Z	The pattern does not match sample timestamps consistently.	Prefer OffsetDateTime / Instant with JavaTimeModule and verify ISO output.
Setter-based and builder-based examples in one method	Same variable names and incomplete first path make the sample hard to compile.	Choose one approach per example; show setter and builder in separate methods/classes.

9. Real-world scenarios

Scenario A: Create a product with a category

An e-commerce platform receives product data from an admin portal. A product requires title, slug, price, category and image URLs. The automation test should validate both the product and the nested category in the returned response.

- Positive checks: creation response status, generated product ID, returned title and price, category ID/name, and expected image count.
- Negative checks: missing title, negative price, empty image list, invalid category ID, duplicate slug if the API enforces uniqueness.
- Data management: use a unique slug per test run where uniqueness is required; clean up created products if the test environment permits it.

Scenario B: Baseline fixture plus overrides

A stable JSON file contains most product fields. One test changes title; another changes price; another changes category. Deserialization reduces duplicate fixture definitions, and toBuilder() expresses the change explicitly.

```
Products baseline = mapper.readValue(fixtureFile, Products.class);
Products discounted = baseline.toBuilder()
    .title("Laptop - Clearance")
    .slug("laptop-clearance-" + System.currentTimeMillis())
    .price(new BigDecimal("29.99"))
    .build();
```

Scenario C: Timestamp contract validation

A service returns creationAt with UTC Z. The test should not convert it to LocalDateTime and accidentally lose timezone meaning. Deserialize it as OffsetDateTime or Instant, then assert it can be parsed and is present.

Scenario D: Image array behaviour

A product can have multiple images. Test that the response includes expected URLs and the list length is correct. If order matters in the API contract, assert order. If order does not matter, compare as a set or use matchers that reflect the contract.

Scenario E: Detect bad request serialization

If a field unexpectedly becomes null, a timestamp becomes a number, or a property name changes, inspect the serialized JSON before calling the endpoint. A focused unit-level assertion on the JSON can pinpoint serialization issues earlier than a full integration test.

10. Best practices

- Model the actual API contract, not an imagined payload. Match field names, nullability, required fields and data types.
- Use BigDecimal for monetary calculations or when exact decimal representation matters. Avoid float for prices.
- Use OffsetDateTime or Instant when the API timestamp includes Z or an explicit offset. Use LocalDateTime only when the contract has no offset.
- Keep the POJO classes in a model or dto package. Use distinct names such as ProductRequest and ProductResponse if request and response fields differ.
- Use @Builder for readable multi-field construction; do not add it everywhere without a readability benefit.
- Keep sample fixtures under src/test/resources/payload, not mixed with production code.
- Use ObjectMapper consistently. Configure the time module once in a shared test utility instead of rebuilding different mappers with different behavior.

- Validate status code and important business fields. Logging the body alone is not a test assertion.
- Use unique test values for fields such as slug or email when the API enforces uniqueness.
- Log only what helps diagnosis. Avoid logging credentials, access tokens, private user data or large payloads in CI logs.
- Clean up test-created records where possible; tests should not depend on the existence of hard-coded IDs.
- Keep the test deterministic. Do not depend on the current date/time unless the test specifically covers time behaviour.
- Use a test environment and externalize base URLs. Never run create/update/delete practice scripts against production data.
- Keep date-time format patterns consistent across the model. If custom patterns are required, add a focused serialization/deserialization test for them.

11. Troubleshooting checklist

Symptom	Likely cause	What to check
NullPointerException on category	Nested Category object was never created.	Build Category separately or use Category.builder().build().
No setter / method not found	Typo or wrong field name; Lombok processing not configured.	Match exact field names; check Lombok dependency and IDE annotation processing.
Cannot deserialize LocalDateTime	Jackson Java Time module missing or date string has Z/fraction/offset that the type cannot accept.	Register JavaTimeModule; use OffsetDateTime/Instant for offset-aware strings.
Invalid JSON body	Serializer/time config or manual JSON editing issue.	Print payload in a safe test, parse it with ObjectMapper.readTree(), verify JSON field names.
HTTP 400	Payload validation failure or fields do not match API contract.	Inspect error response, required fields, data types and enum constraints.
HTTP 401 / 403	Authentication or authorization problem, not generally a builder problem.	Check credentials/token, scopes and roles.
HTTP 404	Wrong endpoint or resource/route does not exist.	Verify URL, environment and API documentation.
Product price assertion fails	Wrong type or API transforms/rounds price.	Inspect response contract; use BigDecimal or the correct numeric matcher.
Duplicate slug on repeated runs	Test reuses a unique field that must be unique.	Generate a unique slug and clean up after test.
Request uses timestamp array/number	ObjectMapper serializes dates as timestamps.	Register JavaTimeModule and disable WRITE_DATES_AS_TIMESTAMPS if contract needs ISO strings.

12. Hands-on exercises

Attempt each task before checking the answer key. Use your own Maven project and the payload structure from the class source. Treat public endpoint behavior as something to verify, not as guaranteed by this workbook.

Exercise 1 - Build a Category

Create a Category using the Builder with id 10, name Books, slug books and an image URL.

Exercise 2 - Build a Product

Create a Products object with title Java Testing, price 39.90, slug java-testing, a category, and two image URLs.

Exercise 3 - Serialize

Convert the object from Exercise 2 into a JSON string with ObjectMapper.

Exercise 4 - Deserialization

Read product.json from src/test/resources/payload into Products.

Exercise 5 - Copy with toBuilder

Change the title and price on a deserialized product without losing its category and images.

Exercise 6 - Find a null bug

Explain why products.getCategory().setName("Electronics") can fail on a newly constructed Products object.

Exercise 7 - Fix timestamp model

Change a timestamp example ending in Z to use the correct Java time type.

Exercise 8 - API assertions

Write assertions for status 201, product title, non-null id and a positive price.

Exercise 9 - Negative payload

Create a product with price -1 and explain what response you expect according to the API contract.

Exercise 10 - Test isolation

Make slug unique per execution and describe how you would clean up the created test record.

13. Answer key / reference solutions

Exercise 1 - Category builder

```
Category category = Category.builder()
    .id(10)
    .name("Books")
    .slug("books")
    .image("https://shop.example.test/images/books.png")
    .build();
```

Exercise 2 - Product builder

```
Products product = Products.builder()
    .title("Java Testing")
    .slug("java-testing")
    .price(new BigDecimal("39.90"))
    .description("API testing guide")
    .category(category)
    .images(List.of("https://shop.example.test/a.png",
                    "https://shop.example.test/b.png"))
    .build();
```

Exercise 3 - Serialize

```
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
String json = mapper.writeValueAsString(product);
```

Exercise 4 - Deserialize

```
File file = new File("src/test/resources/payload/product.json");
Products product = mapper.readValue(file, Products.class);
```

Exercise 5 - Change selected fields

```
Products revised = product.toBuilder()
    .title("Java API Testing - Updated")
    .price(new BigDecimal("34.90"))
    .build();
```

Exercise 6 - Null bug

products.getCategory() can be null because the Products instance does not automatically instantiate its category field. Construct and assign a Category first, or use nested builders.

Exercise 7 - Date/time

```
OffsetDateTime time = OffsetDateTime.parse("2026-09-29T09:08:39.000Z");
```

Exercise 8 - Assertions

```
response.then().statusCode(201)
    .body("id", notNullValue())
    .body("title", equalTo("Java Testing"))
    .body("price", greaterThan(0));
```

The exact expected success code and response shape must come from the API contract. Use the matching Hamcrest number type for your response mapping.

Exercise 9 - Negative payload

A well-validated product API should normally reject a negative price with a client-error response such as 400 or 422, but the actual expected code must be specified by the API contract. Assert the error structure as well as the status code.

Exercise 10 - Test isolation

```
String uniqueSlug = "java-testing-" + UUID.randomUUID();
// Use uniqueSlug for the test request, capture the created id,
```

```
// and delete the test record in an @AfterMethod/@AfterEach cleanup
// when deletion is supported in the test environment.
```

14. Interview revision questions

What is the Builder pattern?

A readable way to construct an object through named chained methods, ending with build().

Does @Builder convert Java into JSON?

No. It builds the Java object. Jackson serialization or Rest Assured object mapping performs the conversion to JSON.

What is the difference between serialization and deserialization?

Serialization turns an object into JSON; deserialization maps JSON to an object.

Why build Category before Products?

The Products category property must reference an actual Category object; otherwise it may be null.

When is LocalDateTime unsuitable?

When the input carries a timezone/offset such as Z, because LocalDateTime does not preserve that offset.

What does List<String> become in JSON?

A JSON array of string values.

Why use BigDecimal for price?

It avoids common binary floating-point precision surprises in decimal monetary values.

What is ObjectMapper?

Jackson's main API for mapping JSON to Java objects and Java objects to JSON.

Why assert API response fields after POST?

A request being sent successfully does not prove that the server created the expected resource.

Why not hard-code IDs?

IDs can vary across environments and runs; capture the created resource ID from the response.

Final checklist before you run the test

☐ POJO fields match the request contract.

☐ Nested objects are initialized and attached.

☐ Builder methods are generated by Lombok and annotation processing works.

☐ Jackson JavaTimeModule is registered when using java.time fields.

☐ The chosen Java time type correctly represents the API timestamp.

☐ The serialized JSON field names and formats match the expected payload.

☐ The request uses ContentType.JSON and the correct URL.

☐ Assertions verify the agreed success code and business fields.

☐ Credentials and sensitive response data are not exposed in routine logs.

☐ Created test records are unique and cleaned up where possible.

Takeaway A maintainable API test separates concerns: POJOs model the contract, Builder creates the test data, ObjectMapper converts JSON and Java objects, Rest Assured sends the HTTP request, and assertions prove the API behaved as expected.